Processes



Process Concept



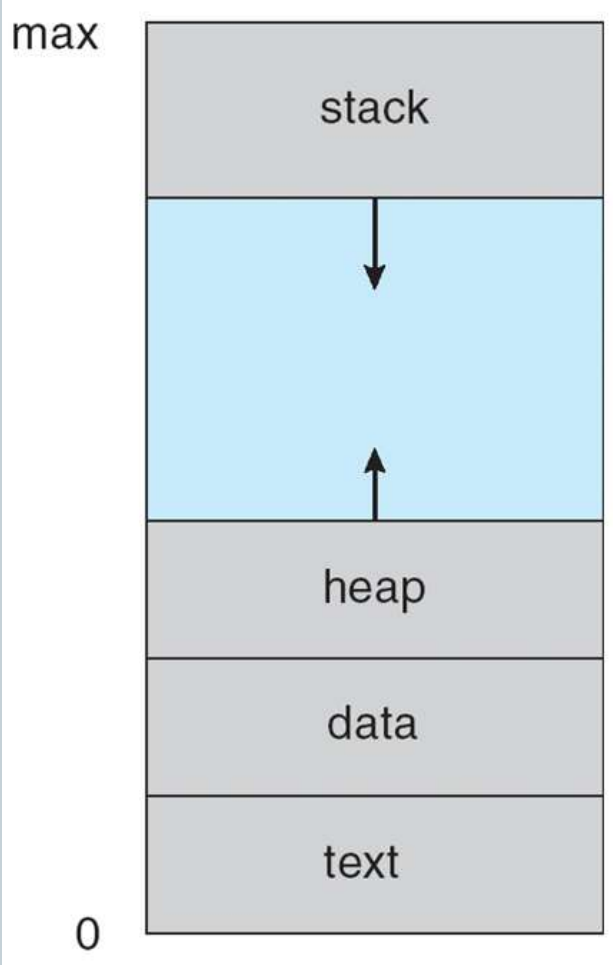
- An operating system executes a variety of programs:
  - Batch system: jobs
  - Time-shared systems :user programs or tasks
- Process : program in execution
- Multiple parts
  - program code: text section
  - Current activity : program counter
  - Data section : global variables
  - Heap : dynamic memory allocation during run time

Process Concept

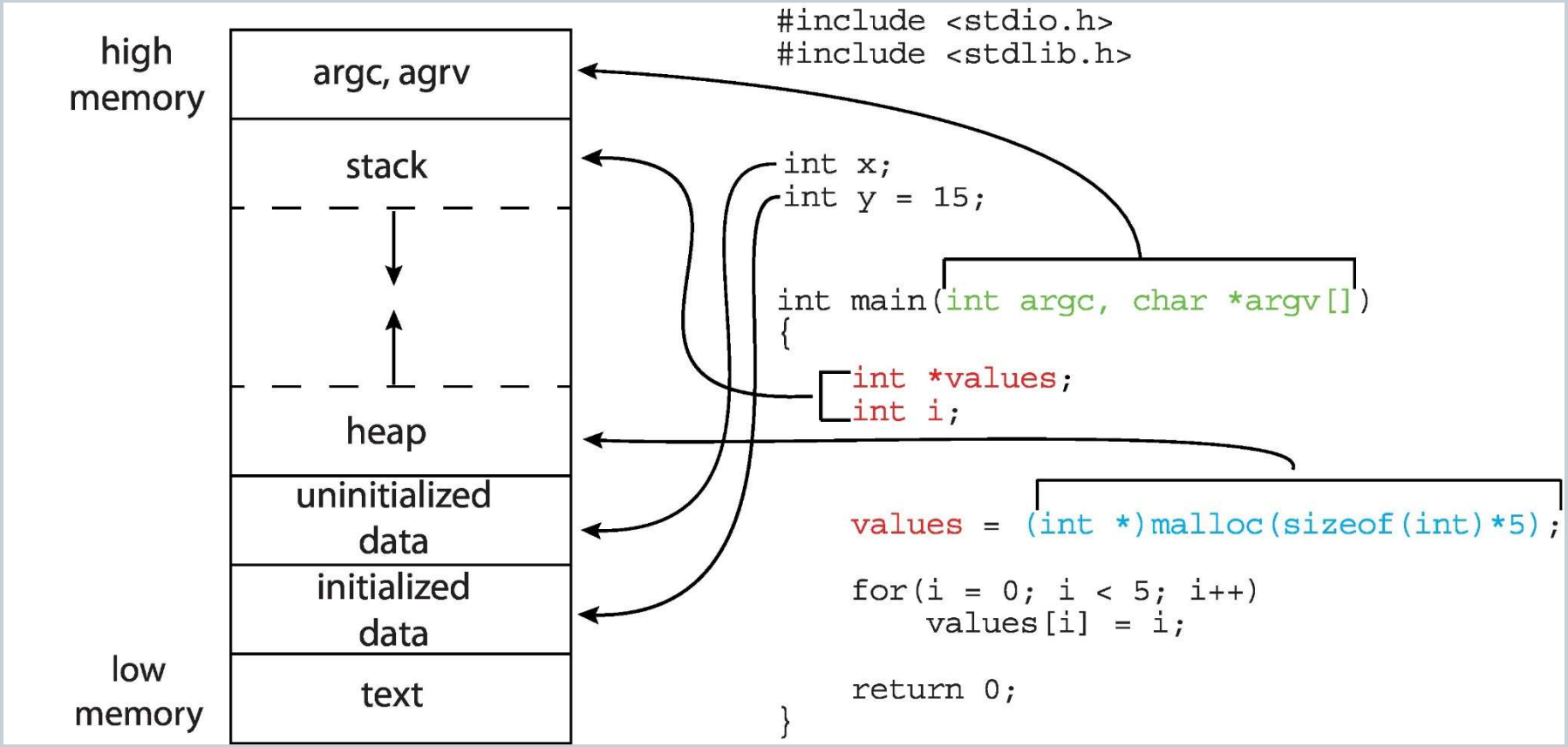


- Program is ***passive*** entity
  - stored on disk (**executable file**)
- One program can be several processes

Process in Memory



Memory Layout of a C Program

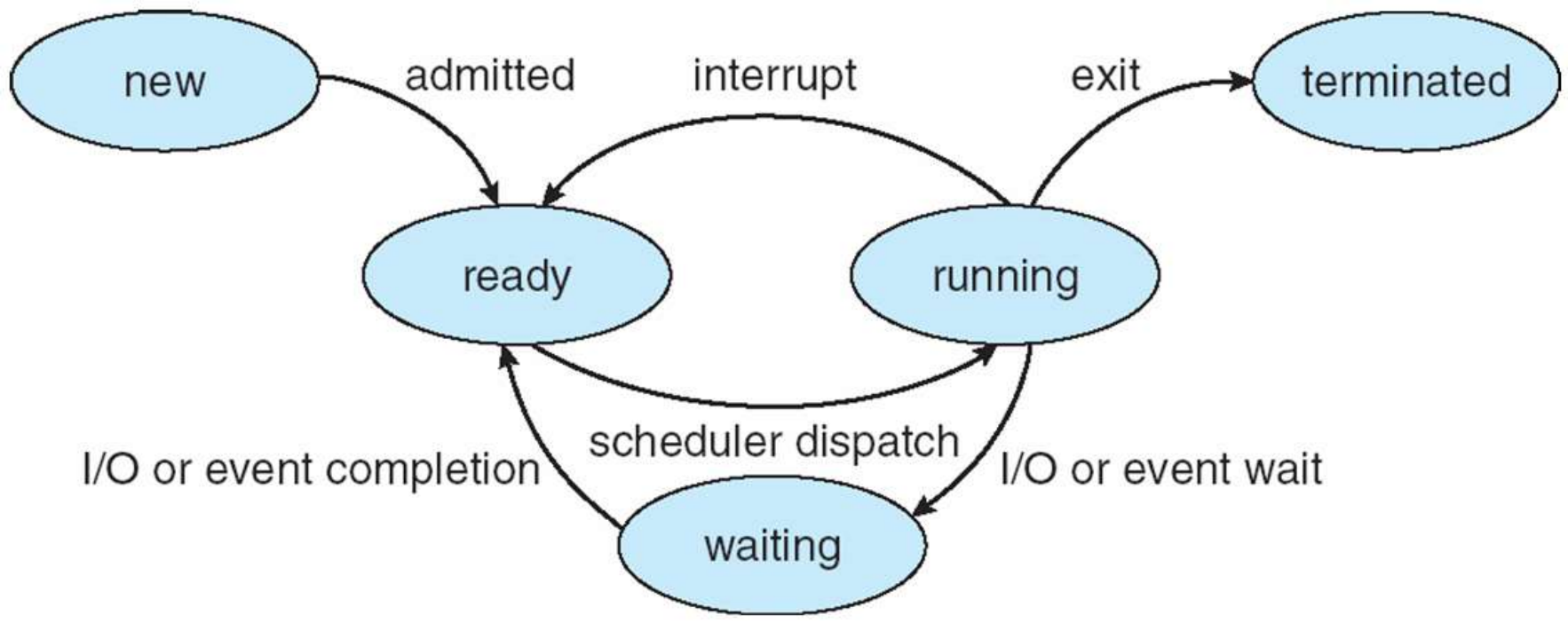


Process State

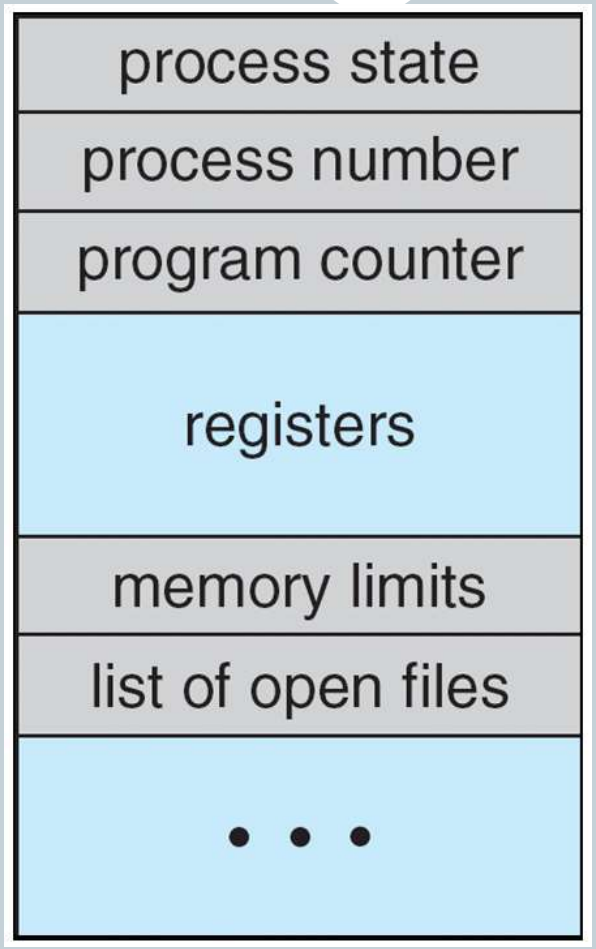


- process change its states
  - **new**: The process is being created
  - **running**: Instructions are being executed
  - **waiting**: The process is waiting for some event to occur
  - **ready**: The process is waiting to be assigned to a processor
  - **terminated**: The process has finished execution

Diagram of Process State



Process Control Block (PCB)



Process Control Block (PCB)



- **Information associated with each process**
  - **Process state : running, waiting, etc**
  - **Program counter :location of instruction to next execute**
  - **CPU registers: contents of all process-centric registers**
  - **CPU scheduling information: priorities, scheduling queue pointers**

Process Control Block (PCB)



- **Memory-management information : memory allocated to the process**
- **Accounting information : CPU used, clock time elapsed since start, time limits**
- **I/O status information : I/O devices allocated to process, list of open files**

Threads



- Multiple program counters per process
- To storage thread details, multiple program counters in PCB

Process Scheduling



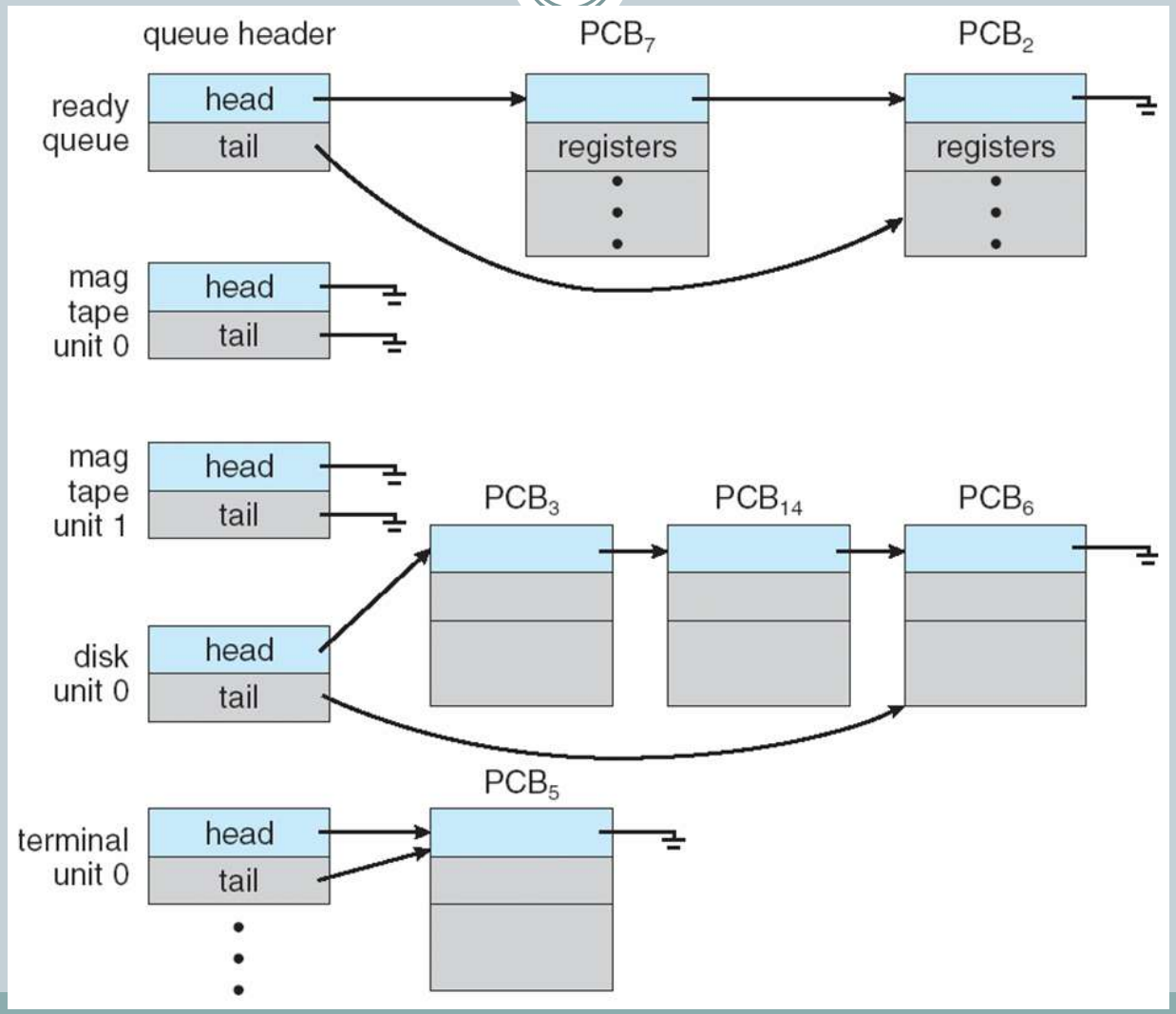
- To Maximize CPU usages
  - quickly switch processes
  - selects processes for next execution

Process Scheduling

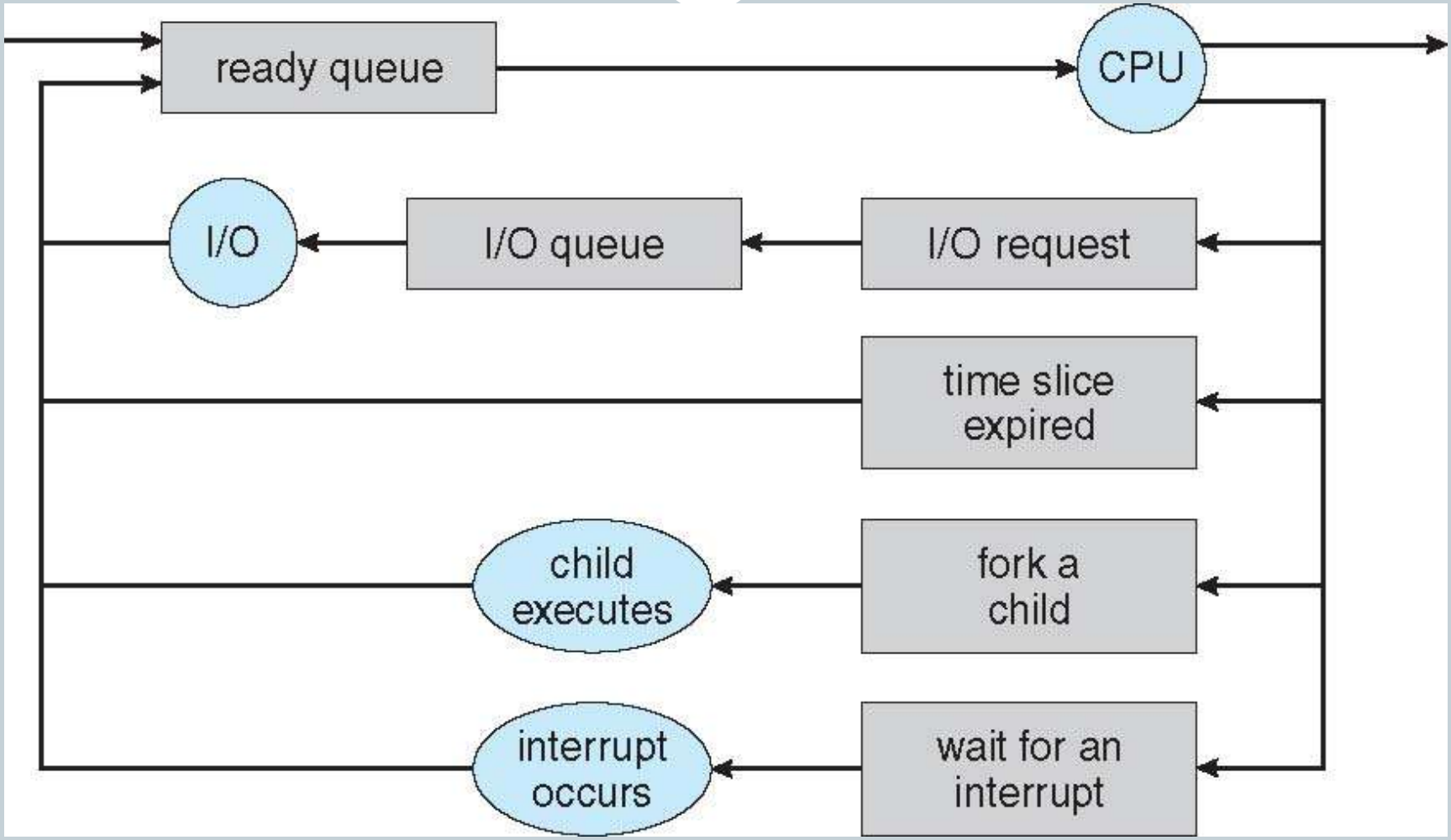


- Maintains scheduling queues
  - Job queue :processes in the system
  - Ready queue : processes residing in main memory, ready and waiting to execute
  - Device queues: processes waiting for an I/O device
- Processes migrate among the various queues

Ready Queue And Various I/O Device Queues



Representation of Process Scheduling

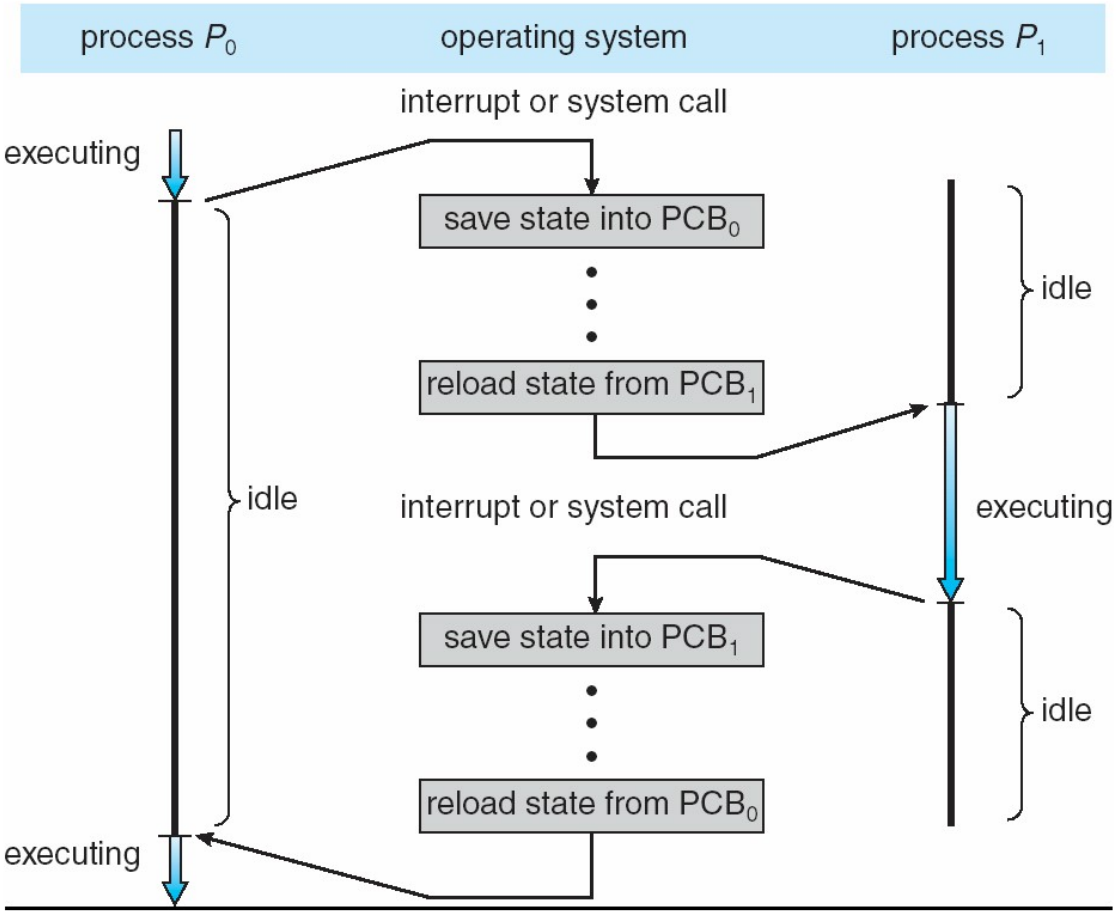


Context Switch



- When CPU switches to another process, the system must **save the state** of the old process and load the **saved state** for the new process

CPU Switch From Process to Process



Schedulers



- **Short-term scheduler** (or **CPU scheduler**) – selects which process should be executed next and allocates CPU
  - Short-term scheduler is invoked frequently (milliseconds) ⇒ (must be fast)
- **Long-term scheduler** (or **job scheduler**) – selects which processes should be brought into the ready queue
  - Long-term scheduler is invoked infrequently (seconds, minutes) ⇒ (may be slow)
  - The long-term scheduler controls the **degree of multiprogramming**

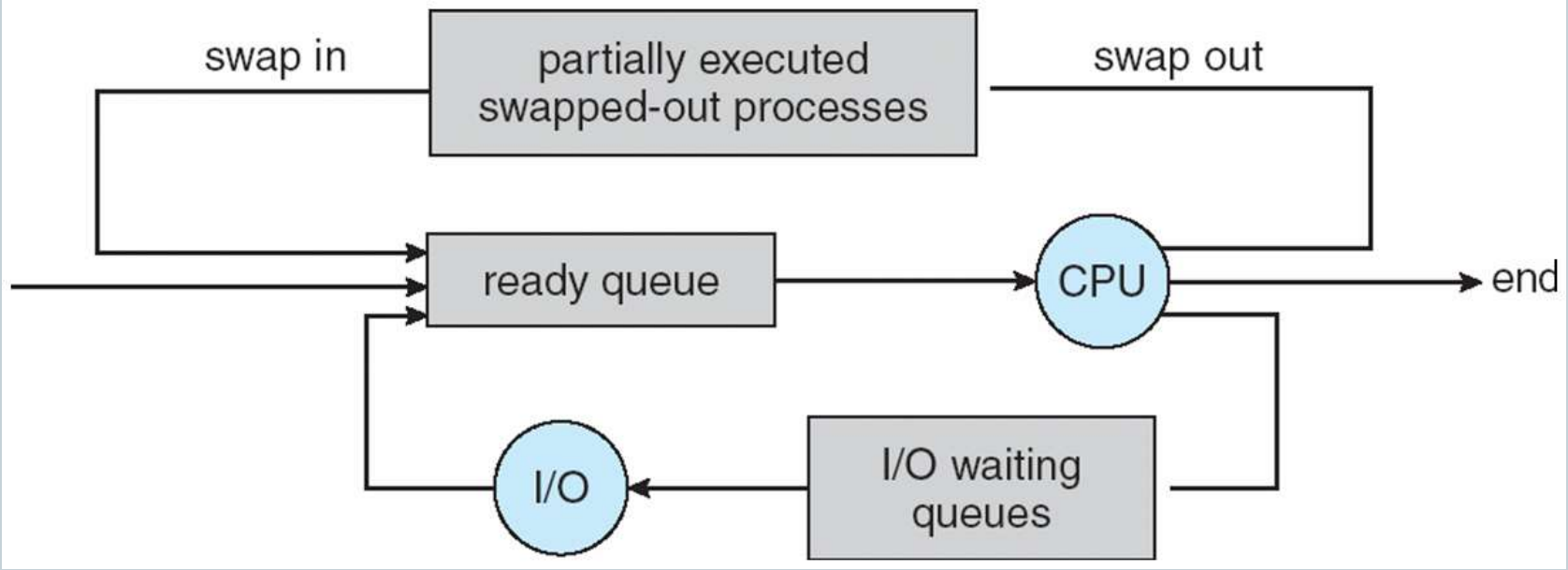
Schedulers



- Processes are described :
  - **I/O-bound process** – spends more time doing I/O than computations, many short CPU bursts
  - **CPU-bound process** – spends more time doing computations; few very long CPU bursts
- Long-term scheduler strives for good ***process mix***

Addition of Medium Term Scheduling

- swapping : Remove process from memory, store on disk, bring back in from disk to continue execution



Multitasking in Mobile Systems



- iOS provides
  - Single **foreground** process: controlled via user interface
  - Multiple **background** processes: in memory, running, but not on the display
  - Limits include single, short task, receiving notification of events, specific long-running tasks like audio playback

Multitasking in Mobile Systems



- Android runs foreground and background
  - Background process uses a **service** to perform tasks
  - Service can keep running even if background process is suspended
  - Service has no user interface, small memory use

Operations on Processes



- process creation
- process termination

Process Creation



- **Parent** process create **children** processes, which, in turn create other processes, forming a **tree** of processes
- Generally, process identified and managed via a **process identifier** (**pid**)

Process Creation



- Resource sharing options
  - Parent and children share all resources
  - Children share subset of parent's resources
  - Parent and child share no resources
- Execution options
  - Parent and children execute concurrently
  - Parent waits until children terminate

Process Termination



- last statement of the process
- asks the operating system to delete it using the **`exit()`** system call.
  - Returns status data from child to parent (via **`wait()`**)
  - Process' resources are deallocated by operating system

Process Termination



- Parent may terminate the execution of children processes using the **`abort()`** system call.
  - Child has exceeded allocated resources
  - Task assigned to child is no longer required
  - The parent is exiting and the operating systems does not allow a child to continue if its parent terminates

Android Process Importance Hierarchy



- Mobile operating systems often have to terminate processes to reclaim system resources such as memory. From **most** to **least** important:
  - Foreground process
  - Visible process
  - Service process
  - Background process
  - Empty process
- Android will begin terminating processes that are least important.

Interprocess Communication



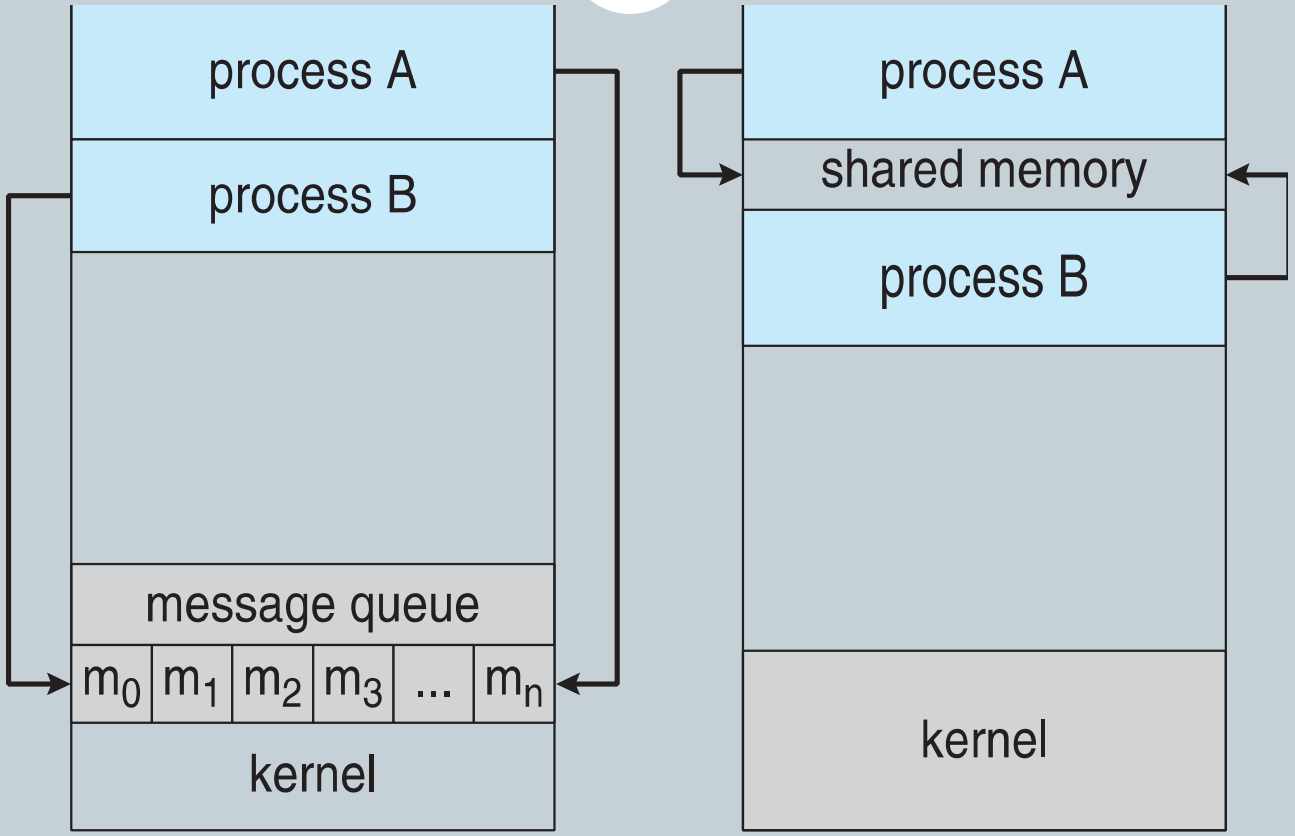
- Processes may be either ***independent*** or ***cooperating***
- Cooperating process can affect or be affected by other processes
- Cooperating processes
  - Information sharing
  - Computation speedup
  - Modularity
  - Convenience

Interprocess Communication



- Cooperating processes need **interprocess communication (IPC)**
- Two models of IPC
  - **Shared memory**
  - **Message passing**

Communications Models



(a) Message passing

(b) shared memory

Communications in Client-Server Systems

- Sockets
- Remote Procedure Calls
- Pipes
- Remote Method Invocation (Java)